ChatGPT

RUM-Informed Synthetic Browser Profiling: Measurement Fidelity, Experimental Validity, and Agent Design

Executive findings and conceptual model

The proposed profiler should be described as **synthetic browser performance measurement against a production target**, optionally with **RUM-informed agent configuration** and **distributed execution**. It is not Real User Monitoring merely because its device, CPU, network, cache, or location parameters are derived from real-user distributions. RUM/field data is collected from actual page loads experienced by real users, whereas laboratory or synthetic data is produced by controlled, predefined test conditions. Google's own terminology treats "lab" and "synthetic" environments as controlled conditions and "field" or RUM data as measurements of actual users, devices, networks, locations, and behavior. [1]

A useful formal model is to distinguish the **source of the workload/configuration** from the **source of the measurement**.

Let a page-performance outcome be represented conceptually as

$$Y = f(W,\ D,\ C,\ N,\ G,\ B,\ S,\ U,\ T,\ \epsilon)$$

where:

- W is the deployed website and server/CDN/third-party system;
- D is device/rendering configuration such as viewport, DPR, input modality, and form factor;
- C is compute capability, including CPU/GPU/memory characteristics;
- N is network behavior and the actual path;
- G is the measurement vantage and geographic/network location;
- B is browser version, engine, flags, and other browser configuration;
- S is browser state: caches, service workers, persistent storage, connections, and navigation history;
- U is actual or scripted user behavior;
- T represents time-dependent production state, deployment version, CDN state, load, experiments, and third parties;
- ϵ represents residual run-to-run variation.

RUM samples this system under the naturally occurring joint distribution of those factors among real visits. Field data is therefore a distribution rather than one canonical "real-user" value. CrUX, for example, aggregates distributions from qualifying real Chrome page loads and exposes histograms, fractions, and percentiles. [2]

A synthetic experiment instead chooses or fixes some vector $x = (D, C, N, G, B, S, U)$ and repeatedly observes the production system under that treatment. Synthetic testing is specifically valuable because variables can be restricted and repeated; geo-distributed scripted monitoring remains synthetic because the clients and scenarios are controlled rather than naturally occurring user visits. [3]

This distinction leads to the central conclusion of the research:

> **RUM evidence can improve the population relevance of synthetic conditions without changing the measurement modality.**

In particular, RUM can answer questions such as “What device classes occur?”, “What RTT distribution do users encounter?”, “How common are phone versus desktop page loads?”, and “How frequently are navigations served through HTTP cache, bfcache, or ordinary navigation?” Those empirical distributions can replace arbitrary synthetic presets. Current CrUX exposes phone/tablet/desktop form factors, an RTT metric, performance histograms and percentiles, and navigation-type fractions; country segmentation is available through BigQuery. [4]

However, **matching individual RUM marginals does not establish that an agent represents a real user**. CrUX explicitly warns that a percentile such as p75 is a statistically derived quantile and need not correspond to the experience of any particular user. Combining an independently selected p75 CPU, p75 RTT, p75 viewport, and p75 geography can therefore construct a combination that is uncommon or even absent from the real joint population. [5]

The strongest conceptual framework is consequently three-dimensional:

Realism dimension	What it asks	What raises it	What it does **not** establish
Controlled-condition realism	Does the agent faithfully execute the condition it claims to represent?	Calibration against real hardware; calibrated CPU rates; measured network impairments; explicit browser/cache state	That the condition is common among users
Real-network/ server realism	Is the measured production system genuine?	Real production URL, deployed CDN/origin, real third parties, geographically distributed/live paths	That the client or path is representative of the user population
Field-user realism	Does the experiment represent the distribution of actual users and visits?	RUM-derived sampling/weights, preservation of correlations, realistic navigation/state/ behavior, validation against field outcomes	Conversion of synthetic measurements into RUM

These axes are largely independent. A synthetic run can have **high production-system realism but low field-user realism**, for example a scripted Chrome instance in a cloud datacenter requesting the genuine production site. Conversely, a highly calibrated phone/network profile can have good controlled-condition realism while still sampling only one of many production/network states. This interpretation follows directly

from the fact that lab tests deliberately restrict device, network, and geography while field data encompasses their real-world distributions. [6]

The practical implication is that the future profiler should make claims of the form:

> "Under profile P, from vantage V, using browser state S, the production URL produced outcome distribution Y during measurement window T."

A stronger statement such as "this is the experience of a typical mobile user" requires separate evidence that P and V actually represent the target population. A statement such as "75% of real users experience X" requires field evidence or a carefully validated population estimator; it cannot be obtained merely by naming a synthetic profile "p75."

Terminology and the RUM–synthetic distinction

The terminology becomes clearer if several concepts that are often conflated are treated as orthogonal.

Real User Monitoring / field measurement. RUM measures actual users during genuine visits "in the wild." Its principal advantage is ecological or population relevance: the measured distributions include the users' actual devices, networks, geographic locations, browser optimizations, navigation patterns, and other uncontrolled conditions. RUM can also be related to business outcomes occurring in those sessions. Its disadvantages are correspondingly limited experimental control, less diagnostic visibility than a fully instrumented laboratory trace, privacy and sampling constraints, and difficulty isolating causes from correlated conditions. Google summarizes the trade-off as field data capturing real-world experience and enabling KPI correlation, while offering a more restricted metric/debugging surface than laboratory tooling. [1]

Synthetic monitoring. Synthetic measurement uses an artificial or scripted client. The term covers more than Lighthouse: it can include CI tests, scheduled production probes, scripted transactions, load testing, local performance suites, and geographically distributed monitors. Ilya Grigorik's performance-measurement taxonomy explicitly includes "geo-distributed monitoring servers that periodically perform a set of scripted actions" within synthetic testing. [3]

Laboratory performance testing. In current web-performance practice, "lab data" is frequently used approximately synonymously with synthetic data, but a narrower practical distinction is useful. A *lab test* is typically an on-demand controlled experiment used for debugging, optimization, or regression testing, while *synthetic monitoring* often denotes recurring operational probes. Both remain synthetic. Google defines lab data as a page load in a controlled environment with predefined network and device conditions, whose purpose is reproducibility. [7]

Distributed synthetic monitoring. This changes the topology, not the measurement category. The same scripted browser can execute from multiple cities, cloud regions, networks, or physical probes. Distributed synthetic tests are especially useful for DNS/CDN behavior, path latency, regional availability, and detecting geographically localized problems, but the observations remain measurements from the chosen probes rather than samples of naturally occurring users. RIPE Atlas illustrates why vantage matters: network

measurements can be analyzed at IP, ASN, and country level, revealing distinct routes and network-path behavior rather than a simple geographic-distance variable. [8]

Browser/device emulation. Emulation is a mechanism used *inside* a synthetic test. It can alter viewport dimensions, DPR, user-agent/client characteristics, touch/mobile rendering behavior, CPU scheduling, network parameters, or geolocation APIs. It should not be confused with physical-device equivalence. Chrome itself explicitly warns that CPU throttling is relative to the host computer and that DevTools cannot genuinely reproduce a mobile CPU architecture on a desktop machine. [9]

A useful hierarchy for the proposed system would therefore be:

measurement modality → target → agent implementation → configured condition → vantage → browser state → scripted scenario.

For example:

> synthetic measurement → production URL → desktop-hosted Chromium → phone viewport + calibrated mid-tier CPU + empirical RTT → London cloud vantage → clean storage → scripted landing-page load.

That description is substantially more defensible than calling the same run a “mobile RUM simulation.”

Comparative strengths

The complementary nature of RUM and synthetic data is well established. Laboratory tooling offers reproducible execution and deep traces suitable for locating bottlenecks; field data tells you whether those bottlenecks matter under actual user conditions. Google explicitly recommends reasoning about them together rather than expecting either source to replace the other. [10]

Question	RUM/field	Controlled synthetic
What are users actually experiencing?	**Strong**	Only indirectly, to the extent that profiles are representative
What causes a regression?	Often difficult because variables co-vary	**Strong**, if treatment and nuisance variables are controlled
Can it run before production?	No real-user population yet	**Yes**
Can it reproduce an exact device/network/state?	Usually not on demand	**Yes**, subject to fidelity of emulation
Does it capture real user behavior and repeat visits?	**Yes**, within instrumentation/ sample limits	Only scripted approximations

Question	RUM/field	Controlled synthetic
Can it expose detailed traces and request/task timelines?	Usually restricted	**Strong**
Does it naturally sample device/ network/location diversity?	**Yes**	No; diversity must be designed
Can it measure rare or deliberately adverse conditions repeatedly?	Only when users encounter them	**Yes**
Is it naturally causal?	No; ordinary RUM is observational unless embedded in an experiment	Potentially strong if treatment is isolated
Is it automatically population representative?	Potentially, subject to coverage/ sample bias	**No**

These differences also explain why lab and field metrics should not be expected to match numerically. A lab run is typically one point under one device/network/location combination; field data is the distribution over actual conditions and behavior, including bfcache and other browser optimizations. Google explicitly notes that the particulars of a laboratory run "may or may not" represent the field p75. [6]

There is direct empirical evidence of this non-equivalence. An HTTP Archive analysis joining URL-level Lighthouse laboratory data to CrUX field data found only moderate correlation between Lighthouse performance score and the share of "good" LCP experiences ($r \approx 0.491$), weaker correlation for CLS ($r \approx 0.368$), and a problematic negative FID relationship in that historical dataset. The authors themselves warn about causal interpretation and note that the study predates later CLS definition changes; FID has since been replaced by INP as a Core Web Vital. The study is therefore evidence that synthetic and field signals can be directionally related without being interchangeable—not a current calibration equation. [11]

Likewise, field research using CrUX has found systematic differences across device types and countries. Sengupta and colleagues' analysis of CrUX across desktop, phone, tablet, and nine European countries found desktop experiences outperforming phone/tablet measurements across the metrics they studied and substantial country-level differences. These results support stratification by device class and geography, but they do **not** show that a generic "mobile" synthetic preset reproduces those populations. [12]

Fidelity of device, CPU, network, geography, and browser-state emulation

The validity of an agent depends strongly on **which property it claims to emulate**. Viewport emulation, CPU throttling, network shaping, and geographic placement have very different fidelity limits.

Device and mobile/desktop approximation

A phone/desktop distinction is empirically meaningful: CrUX currently exposes `phone`, `tablet`, and `desktop` as separate form-factor dimensions inferred from user-agent information, and published CrUX analysis finds materially different performance distributions among them. [13]

But **form factor is not hardware class**. "Phone" does not tell you the CPU generation, GPU, memory bandwidth, thermal state, storage speed, browser version, radio technology, viewport, or access network. Consequently, an agent should avoid interpreting `PHONE` as a fully specified physical device.

For synthetic purposes it is useful to distinguish three levels:

Responsive/browser emulation changes properties such as viewport, DPR, user-agent behavior, and mobile rendering. This can be highly valid for testing responsive resource selection and layout, but it does not make a desktop processor or GPU equivalent to phone hardware.

Shaped desktop execution adds calibrated CPU and/or network restrictions. This can bring particular computational and network-sensitive workloads nearer to a target class, but remains an approximation.

Scripted physical-device execution uses a genuine handset and browser while retaining a synthetic workload. It offers much higher device-hardware fidelity but is still synthetic rather than RUM because the visit and interaction are generated by the experiment rather than an actual user.

Chrome's own performance guidance recommends real-device debugging when desktop emulation fails to reproduce a field problem, which is an important boundary condition for any emulation-based profiler. [14]

What CPU throttling actually does

Chrome DevTools CPU throttling is not CPU virtualization. At the protocol level, `Emulation.setCPUThrottlingRate` accepts a slowdown factor: 2 means a nominal 2× slowdown, and so forth. [15]

Chrome's engineering explanation is more revealing. With throttling enabled, Chrome creates a separate throttling thread that repeatedly interrupts and suspends the tab's main thread. At 4× throttling, for example, the main thread is suspended for approximately three quarters of the time, causing main-thread work to take approximately four times as long. [16]

This means CPU throttling has relatively good conceptual fidelity for **main-thread computational throughput**—JavaScript execution, style calculation, layout, and other work occurring on that thread. It is much weaker as a model of a complete mobile system.

The slowdown factor is also **hardware-dependent**. Chrome documentation explicitly states that throttling is relative to the computer on which DevTools is running: 2× means twice as slow as *that machine's* normal execution, not twice as slow as a standardized reference processor. Chrome therefore introduced host-specific calibration that benchmarks the local computer and derives "low-tier" and "mid-tier" presets. [17]

That calibration is a significant improvement over universal values such as “4× = mobile,” but it does not solve all fidelity problems. Chrome demonstrates a GPU-heavy interaction taking roughly 270 ms on a CPU-throttled desktop versus roughly 620 ms on a real mid-tier phone because the desktop GPU is not equivalently slowed. Chrome also identifies problematic cases involving deep dependency chains, simulated networking, IPC, disk access, and memory caches. [14]

The defensible interpretation for profiler agents is therefore:

> **A calibrated CPU throttle represents a computational-throughput class for workloads dominated by the resources the throttle actually affects; it does not represent a complete physical phone model.**

Low-/mid-/high-end labels can still be operationally useful if they are defined by explicit calibration targets rather than marketing terminology. For example, a “compute tier B” profile could mean “host-specific throttle calibrated to match the median main-thread benchmark of reference devices X–Y,” accompanied by the calibration version and error against physical reference phones.

This distinction is supported by field research beyond Chrome’s own experiments. Steiner and Gao, using real-user performance data, found that smartphone/device generation and processing capability could materially affect page-loading performance, reporting up to a 56% reduction in median page-load time between device generations in their dataset. That evidence argues against treating network class as the sole determinant of “mobile performance.” [18]

Network throttling is not one technique

Three network modes should be distinguished explicitly in the profiler.

Browser/DevTools request-level throttling. Chrome DevTools Protocol exposes minimum request-to-response-header latency plus aggregate download and upload throughput controls. Lighthouse documentation describes this as request-level throttling and specifically cautions that real mobile latency acts at packet level, making request-level shaping an imperfect approximation. [19]

Packet/network emulation. Operating-system or network-appliance mechanisms operate below the browser. Linux `netem`, for example, can introduce packet delay, jitter, loss, reordering, duplication, and rate constraints. Lighthouse documentation characterizes packet-level shaping as capable of more accurate network simulation than request-level throttling, although this increased realism can also increase measurement variance. [20]

Observed live-path measurement. No attempt is made to impose a synthetic connection; the browser simply uses its real path from the selected probe to the target. This is the highest-fidelity measurement of **that particular path at that particular time**, but it has reduced experimental control and may be poorly representative of end users unless the probe's ASN, access technology, resolver behavior, and routing resemble theirs. RIPE Atlas exists precisely because network paths and connectivity differ across IPs, ASNs, and countries. [8]

A fourth mode deserves separate treatment because Lighthouse commonly uses it:

Model-based simulation. Lighthouse's default simulated throttling predicts how an observed execution would behave under specified constraints rather than necessarily running the entire page under those constraints. Lighthouse warns that the "original trace" therefore does not have the same metric values as the simulated result. Its Lantern documentation reports that, in its evaluation, Lantern was around 6–13% less accurate than DevTools throttling, while individual DevTools-throttled observations themselves exhibited substantial variance. Those figures are specific to that evaluation, but they strongly support repeated measurement rather than single-run decision-making. [21]

This taxonomy matters for the user's production-target scenario. A DevTools-throttled browser still communicates with the genuine production CDN/origin over a real underlying path, but the effective browser-observed conditions include both real-path behavior and artificial browser constraints. Packet shaping similarly overlays configured impairments on a physical route. A non-throttled distributed agent observes the actual path but sacrifices the controlled network treatment.

Generic cellular labels are especially problematic as population claims. CrUX removed its Effective Connection Type dimension in February 2025 and replaced it with an RTT metric; the former ECT categories were broad enough that most sufficiently fast connections collapsed into "4G." [22]

That change reinforces an important design principle:

> A profile labeled **"RTT p75 = 180 ms, downstream distribution class B"** has clearer empirical semantics than one labeled **"Slow 3G."**

"Slow 3G" remains useful as a deterministic stress-test preset, but it should not be presented as representative unless evidence shows that its parameters correspond to a defined user segment.

Empirical cellular research further cautions against reducing networking to static RTT/bandwidth numbers. Rajiullah and colleagues analyzed nearly two million web browsing sessions in operational mobile environments, demonstrating the usefulness of studying web performance across real commercial mobile networks rather than assuming a single generic cellular condition. [23] Research on HTTP/2 over lossy cellular conditions has likewise used observed cellular behavior as the basis for controlled emulation, illustrating an important methodology: **measure field path characteristics first, then reproduce selected characteristics experimentally**. [24]

Geography and distributed agents

Geographic execution is valuable, but "country" or "city" should not be treated as the network treatment itself. A browser in an AWS region and a consumer handset in the same city may use different ASNs, resolvers, CDN mappings, peering links, congestion points, and last-mile technologies.

RIPE Atlas network-path tooling explicitly represents measurements by IP, ASN, and country and can expose path patterns and anomalies. This supports treating a synthetic vantage as a tuple more like

$$G = (\text{country}, \text{metro}, \text{ASN}, \text{access type}, \text{resolver/path context})$$

rather than a pair of latitude/longitude coordinates. [8]

Distributed synthetic agents are consequently strong for questions such as:

- Does this production deployment perform differently from several stable network vantages?
- Does a CDN or DNS configuration expose regional regressions?
- How sensitive is performance to measured RTT/path variation?
- Is a regression globally reproducible or localized?

They are weaker for the claim “this represents users in Sri Lanka/Germany/Brazil” unless probe networks have been selected to resemble the access networks actually used by those users. CrUX can provide country-level field evidence through BigQuery, but the public CrUX API does not expose country as a query dimension. [25]

Cache, repeat visits, service workers, and “warm” state

A simple cold/warm boolean is insufficient.

HTTP caching alone materially changes network work: RFC 9111 specifies that a fresh cached response can be reused without contacting the origin for validation, reducing latency and network overhead; stale responses can lead to conditional validation. [26]

Service-worker state is separate. The Service Workers specification explicitly states that Cache API objects are **not** part of the browser's HTTP cache; authors manage them independently, and service-worker registrations persist beyond individual page/client lifetimes. Clearing the HTTP cache therefore does not necessarily return the browser to a truly first-visit state. [27]

A defensible browser-state model should therefore record several independently configurable properties:

HTTP cache state; service-worker registration/control state; Cache Storage; cookies and origin storage; navigation/history state; and, where the test harness permits, connection/session state.

A profile such as “warm” should state exactly which of these are preserved.

CrUX provides unusually useful field evidence here. Since March 2024 it reports fractions for navigation types including ordinary navigation, main-document cache navigation, reload, restore, back/forward, bfcache, and prerender. Google also stresses that these navigation types materially affect performance. Public CrUX cannot break LCP or other metrics down *by* navigation type, however, so first-party RUM is needed for that conditional analysis. [28]

This suggests at least three distinct synthetic state classes rather than one cold/warm toggle:

new/clean browser, where relevant origin state and caches are absent;

returning browser, where defined persistent state and caches have been established through a preparatory journey;

history/navigation-state scenarios, covering bfcache/back-forward/reload behavior where the application supports them.

Those can later be assigned empirical weights from first-party RUM or CrUX navigation fractions rather than assuming, for example, that every benchmark represents a first visit.

Deriving defensible RUM-informed agent profiles

The public data now supports a substantially more empirical approach than fixed labels such as “mobile / 4× CPU / Slow 3G.”

CrUX exposes form-factor fractions and conditional queries, RTT histograms/p75, URL- or origin-level performance distributions, and navigation-type fractions. Its API is based on a rolling 28-day aggregate, while BigQuery provides the historical dataset back to 2017 and supports country-level analysis. [29]

The following agent dimensions have differing levels of empirical support:

Candidate dimension	Evidence basis	Synthetic implementation	Validity assessment
Phone / tablet / desktop share	CrUX form-factor fractions and first-party RUM. [30]	Viewport/browser profile plus separate compute treatment	**Strong for stratification; weak as complete hardware model**
Viewport / DPR	First-party RUM; responsive design telemetry	Browser metrics override	**Strong for responsive layout/resource-selection questions**
CPU / compute tier	First-party device data, reference-device benchmark corpus, Chrome calibration	Host-specific calibrated slowdown	**Moderate**; main-thread approximation, host/GPU limitations. [31]
RTT	Current CrUX RTT histograms/p75; first-party RUM; active path measurements	DevTools delay, packet shaping, or vantage selection	**Strong empirical dimension; implementation fidelity depends on method**. [32]
Bandwidth	First-party telemetry, controlled active measurements, network datasets	Browser throughput cap or packet shaper	**Moderate**; throughput is dynamic and path/ application dependent
Loss/jitter/ reordering	Active/mobile-network traces rather than ordinary CrUX	Packet-level emulator	**Potentially strong when trace-derived; weak when arbitrary**. [20]
Country/ region	CrUX BigQuery and first-party RUM	Distributed agent placement	**Strong as a population stratum; geography alone is not path fidelity**. [33]

Candidate dimension	Evidence basis	Synthetic implementation	Validity assessment
ASN/access-network class	First-party network metadata or RIPE-style measurements	Probe placement in selected networks	**High relevance for path realism**. [8]
Navigation/ cache state	CrUX navigation fractions and first-party RUM	Prepared clean/ returning/bfcache scenarios	**Strong if state is explicitly defined**. [34]
Browser/ engine/ version	First-party RUM	Native browser or browser-farm selection	**Important where target population is multi-browser**; CrUX itself is Chrome-specific
User interaction path	First-party RUM/session-level interaction evidence	Deterministic scripted flow	**High internal control, but population realism depends on script derivation**
Production variant	Response headers, application telemetry, deployment metadata	Record rather than emulate	**Critical nuisance variable for production experiments**

Several profile-construction methodologies are defensible, with different objectives.

Quantile profiles choose interpretable conditions such as phone + median RTT, phone + p75 RTT, and phone + tail RTT. This is simple and reproducible. Its major weakness is that independently choosing quantiles across dimensions can destroy correlations. CrUX's warning that a percentile is a derived value and need not correspond to an actual user is directly relevant here. [5]

Stratified profiles first divide the field population into meaningful strata—for example phone/desktop and geography—and derive conditional network and state distributions within each stratum. This is preferable to mixing global marginals because phone users, desktop users, and countries have demonstrably different performance distributions. [12]

Joint-distribution clustering or medoid profiles can be built from first-party RUM records when richer telemetry is available. Rather than independently selecting "typical" values, cluster actual observed tuples and select a real or near-real medoid as the synthetic representative. Each profile receives the traffic weight of its cluster. This is an analytical recommendation rather than a property of CrUX: public CrUX aggregation does not expose every user-level joint tuple needed for such clustering.

Tail profiles intentionally represent disadvantaged conditions. They should be labeled by their empirical selection rule—"slowest RTT decile among phone sessions," for example—rather than "typical low-end mobile." Their purpose is robustness testing, not central-tendency prediction.

Observed-path profiles select real distributed vantages and avoid synthetic browser network throttling. These sacrifice network repeatability in exchange for measurement of genuine changing paths.

Trace-informed emulated profiles derive loss/delay/jitter characteristics from measured networks and replay a controlled approximation through a packet emulator. Research that observes cellular behavior and then emulates selected characteristics provides precedent for this field→lab methodology. [24]

A particularly important design principle is to separate two validation questions:

> **Profile-definition validity:** Is the specified condition actually supported by evidence about the target user population?
>
> **Profile-execution validity:** Does the synthetic browser reproduce that specified condition accurately?

RUM primarily improves the first. Real-device calibration, packet emulation validation, probe placement, and browser instrumentation improve the second. Neither alone establishes the other.

Chrome is already moving in this direction operationally. Its Performance panel can use CrUX field data to recommend throttling conditions based on the site's users, including its field p75, while its CPU calibration derives host-specific low-/mid-tier throttling rates. This is effectively an industry precedent for **field-informed laboratory configuration**, while remaining explicitly laboratory measurement. [35]

Internal and external validity of production-target synthetic experiments

The key methodological advantage of your design is that it can combine **controlled client-side treatments** with **genuine deployed infrastructure**. The key methodological risk is that those same production components are themselves changing experimental variables.

Internal validity

Internal validity asks whether an observed difference can actually be attributed to the factor under investigation.

For a profile-comparison experiment, the ideal estimand might be:

$$\Delta = E[Y \mid P_B, W] - E[Y \mid P_A, W]$$

where the website W is held stable and only the profile changes.

On a real production URL, W is never perfectly stable. CDN edge state, origin load, third-party responses, advertisements, consent flows, feature experiments, personalized content, resource versions, and network routes can vary between runs. Synthetic control therefore improves internal validity only if those nuisance variables are measured, held constant where possible, or balanced across treatments.

A robust experiment should control or explicitly record at least:

Element	Why it matters	Experimental treatment
URL and page journey	Different routes/resource graphs are different workloads	Fixed, versioned script
Browser build/version	Rendering, scheduling, networking, metrics can change	Pin and report
Browser flags/ extensions	Can alter execution and network behavior	Clean standardized profile
Viewport/DPR/input/ UA	May change responsive resources and content	Fixed within treatment
Host hardware and host load	CPU throttle is relative to host	Dedicated workers; calibrated host fingerprint. [9]
CPU throttle/ calibration	Determines synthetic compute treatment	Record factor and calibration benchmark
Network method	Simulation, request throttling, packet shaping and live path answer different questions	Do not silently mix modes. [36]
Probe/vantage/ASN	Changes path and CDN/DNS behavior	Block or stratify by vantage. [8]
HTTP cache	Can eliminate requests or validation	Explicitly initialized. [26]
Service-worker/Cache Storage	Independent of HTTP cache	Explicitly register/clear/ prepare. [27]
Cookies/storage/ consent state	Can change page content and third parties	Seed or clear deterministically
Test order and time	Production conditions drift	Interleave/randomize profile order within short blocks
Site deployment/ experiment variant	Changes application treatment	Capture build/version/ experiment metadata where possible
Third-party outcomes	Often nondeterministic	Record request/response evidence and use repeated trials
Instrumentation settings	Tracing itself may alter execution	Identical instrumentation across compared conditions

Multiple runs are essential. Lighthouse's own Lantern evaluation provides a concrete example: single DevTools-throttled observations in its dataset had standard deviations on the order of 9–13%, demonstrating that single-run differences can be dominated by run variance in at least some conditions. [37]

For comparison experiments, **interleaving** is preferable to running every A trial and then every B trial. An A–B–B–A or randomized block schedule makes temporal production variation less likely to be mistaken for a profile effect. Vantage should similarly be treated as a block or modeled level rather than pooling every run as though measurements were independent.

The profiler should also distinguish three causal questions:

Website effect: compare versions/sites while keeping profile constant.

Profile effect: compare conditions while keeping site and scenario constant.

Website × profile interaction: determine whether an optimization benefits one user condition differently from another.

When CPU, RTT, bandwidth, viewport, and cache state all change simultaneously, the result identifies the effect of the **composite profile**, not the independent causal contribution of each component. Component attribution requires one-factor-at-a-time or factorial experimental designs.

Reproducibility is especially important in web measurement because small setup changes can alter outcomes. The web-measurement literature has explicitly studied reproducibility and replicability problems across published measurement studies, reinforcing the need to report browser configuration, target selection, timing, measurement topology, and other experimental details rather than treating “Chrome/Lighthouse” as a complete methodology. [38]

External validity

External validity asks what larger population or setting the results can legitimately describe.

A highly controlled synthetic agent hitting production has **strong direct validity for its own configured condition**:

> “This is what the real deployed site did under these specified browser, client, state, vantage, scenario, and time conditions.”

Repeated across several vantages, it can support:

> “The effect was robust across these tested profiles and vantages.”

If the profiles are selected from empirical field strata, another justified statement is:

> “These tests intentionally cover conditions observed among the site's real-user population.”

What does **not** automatically follow is:

> “This is what real users experience overall.”

Field experience includes the naturally occurring diversity of hardware, networks, browser states, navigation types, behavior, background load, and geography that a synthetic experiment constrains. Google explicitly identifies this distinction as the reason lab and field values differ. [6]

A population estimate constructed from synthetic profiles,

$$\hat{Y}_{population} = \sum_k w_k \hat{Y}_k,$$

can be scientifically useful, but its validity depends on several assumptions: the weights w_k must accurately represent the intended field population; the profiles must preserve important correlations among conditions; omitted variables must not be major effect modifiers; and the synthetic implementation must faithfully execute each profile. Those are modeling assumptions, not properties conferred by the synthetic tool.

This is particularly important for Core Web Vitals. Field Core Web Vitals represent distributions over real page views. A scripted synthetic interaction path does not automatically reproduce field INP, because real users perform different interactions at different times and may generate different interaction distributions. Likewise, a synthetic cold navigation cannot reproduce a field distribution containing bfcache, cached documents, and prerendered loads. CrUX explicitly shows that such navigation classes occur and affect reported field performance. [39]

The production URL nevertheless gives the design substantial **system realism**. Unlike a local fixture, it exercises genuine deployed origin resources, CDN behavior, and third-party services as they exist during the experiment. That makes it particularly appropriate for operational questions such as regression detection, CDN/path sensitivity, third-party impact, production optimization verification, and controlled comparisons across client conditions. It simultaneously means production nondeterminism becomes a threat to causal attribution and must be handled experimentally.

The right summary is therefore:

Production target ≠ real user.

Real network path ≠ representative network path.

Mobile viewport ≠ mobile device.

CPU slowdown ≠ physical mobile processor.

RUM-derived parameters ≠ RUM measurement.

But:

RUM-derived parameters + calibrated synthetic execution + genuine production targets can produce substantially more defensible synthetic experiments than arbitrary laboratory presets.

Evidence-supported profiler-agent strategies

No single design should be selected prematurely because the appropriate strategy depends on whether the profiler is optimizing reproducibility, population coverage, live-path realism, or hardware fidelity. Several architectures are evidence-supported.

A small, calibrated canonical-profile library would define a handful of reproducible agents: desktop, representative phone-compute tier, lower-compute phone tier, and perhaps selected network tails. CPU tiers would use host calibration rather than universal throttle multipliers, with physical-device reference tests used to quantify approximation error. This maximizes reproducibility and debugging value while accepting limited population coverage. Chrome's calibrated low-/mid-tier approach provides a practical precedent. [40]

A RUM-stratified weighted profile library would derive profile shares from CrUX or first-party RUM. Phone/desktop shares, conditional RTT distributions, geography, and navigation-state frequencies can determine test allocation and reporting weights. Current CrUX is especially suitable for form factor, RTT, navigation types, and country-level aggregate evidence, although it cannot provide a full device/CPU joint distribution. [41]

A first-party joint-distribution profiler would go further by deriving clusters or medoids from individual RUM observations. This would preserve relationships such as “these network conditions commonly co-occur with this form factor and geography” rather than manufacturing composite quantile users. It offers the strongest route toward synthetic population modeling, but introduces data-volume, telemetry, privacy, and maintenance requirements and still requires a separate mapping from measured field characteristics to executable synthetic settings.

A distributed live-path mode would deliberately turn off artificial network throttling and test from multiple geographic/network vantages. This gives high real-path realism and is ideal for CDN, DNS, regional, and operational monitoring questions. Its outputs should be reported by vantage/ASN rather than advertised as generic “users in region X.” RIPE Atlas path analysis demonstrates why ASN/path information is often more informative than geography alone. [8]

A packet-trace-informed controlled-network mode would collect empirical network distributions or traces and transform them into packet-level impairment profiles. This is attractive for repeatable cellular/path research because packet emulation can represent loss, jitter, and reordering that simple request-level throttles cannot. [20]

A physical-reference-device tier would run selected canonical scenarios on real low-, middle-, and high-performance handsets. These runs remain synthetic but provide a calibration and validation set for desktop-hosted emulation. The physical devices need not run every test; they can establish error envelopes and detect cases where CPU throttling fails because GPU, memory, storage, or architecture dominates. Chrome’s own CPU-throttling analysis gives a concrete example in which desktop CPU shaping substantially underestimates a real phone's GPU-bound interaction time. [14]

A state-aware journey strategy would treat clean, returning, cached, service-worker-controlled, and history-navigation experiences as distinct profiles or phases rather than collapsing them into “cold/warm.”

CrUX navigation fractions can provide empirical weights, while first-party RUM can estimate performance conditioned on those states. [42]

These strategies are compatible. A mature architecture could expose the provenance of every agent setting, for example:

> **Profile source:** first-party RUM, 2026-07-01–2026-07-28
> **Population stratum:** phone sessions, region R
> **Viewport:** empirical medoid
> **CPU mapping:** reference-device tier C, calibrated on host benchmark version X
> **Network:** p50 conditional RTT + packet model derived from trace family Y
> **State:** returning-navigation cluster, HTTP cache seeded, SW controlled
> **Vantage:** ASN/metro V
> **Weight:** 17.4% of eligible observed sessions
> **Execution type:** synthetic Chromium against production
> **Validation error:** measured separately against reference devices/paths

That kind of provenance is more scientifically meaningful than the profile name itself.

An especially useful reporting practice would be to publish results in two layers:

Condition-level results preserve internal validity: distributions, medians, confidence intervals, traces, and regressions for each controlled agent.

Population-model results apply empirical RUM-derived weights and are clearly labeled as modeled/estimated synthetic population outcomes rather than field measurements.

This preserves the major virtue of synthetic measurement—controlled comparison—without overclaiming what RUM-derived configuration provides.

There are also established hybrid methodologies rather than a need to posit a new research gap. Chrome DevTools now explicitly uses field/CrUX data to ground laboratory throttling recommendations; PageSpeed tooling deliberately places field and lab evidence side-by-side; HTTP Archive research has joined CrUX field observations to Lighthouse measurements; and networking research has used measurements of operational cellular conditions to construct controlled laboratory experiments. [43] The proposed profiler can be positioned as another member of this broader **field-informed controlled measurement** family rather than as a novel category of RUM.

Priority original papers, specifications, and data sources

The following sources are the highest-priority material to download, archive, or incorporate into the design bibliography. They have been prioritized for direct relevance to agent semantics, calibration, field distributions, network fidelity, and experimental validity rather than for general web-performance background.

Priority source	Type	Why it matters to the profiler
Google, “Why lab and field data can be different”	Primary methodology documentation	Clearest operational distinction between controlled lab/synthetic measurements and actual-user field/RUM distributions; directly supports the terminology and external-validity model. [44]
Chrome UX Report methodology and API documentation	Primary dataset specification	Defines form factors, URL/origin aggregation, histograms, fractions, percentiles, RTT, 28-day aggregation, and the crucial warning that p75 need not describe any single real user. [45]
CrUX BigQuery dataset/ documentation	Primary field dataset	Historical CrUX dataset back to 2017 and the main route to country-level analysis and larger-scale empirical profile derivation. [46]
CrUX Navigation Types documentation/data	Primary field dataset documentation	Supplies empirical frequencies for ordinary, cached, reload, restore, back/forward, bfcache, and prerender navigations; highly relevant to repeat-visit profiles. [28]
Chrome, “More accurate DevTools performance debugging using real-world data”	Primary browser-engineering source	Probably the single most relevant source for your CPU question: documents host calibration, throttle mechanics, physical-device comparison, GPU failure cases, and CrUX-grounded throttling recommendations. [47]
Chrome DevTools Protocol — Emulation domain	Primary protocol specification	Exact semantics of `setCPUThrottlingRate`; necessary if implementing programmable agent controls. [15]
Chrome DevTools Protocol — Network domain	Primary protocol specification	Exact semantics of DevTools network latency/ throughput controls and request matching; necessary to state what network agents actually do. [48]
Lighthouse throttling documentation	Primary implementation documentation	Explicit distinction between simulated, request-level, proxy, and packet-level network throttling and their fidelity trade-offs. [36]
Lighthouse Lantern documentation and accuracy evaluation	Primary implementation/ evaluation source	Quantifies simulation-vs-DevTools error and single-run variability in Lighthouse's evaluation; supports repeated-run methodology. [37]
RFC 9111, HTTP Caching	Standards-track specification	Canonical semantics for fresh/stale cache reuse, validation, and why repeat visits differ from cold loads. [26]

Priority source	Type	Why it matters to the profiler
W3C Service Workers specification	Web-platform specification	Establishes persistent service-worker state and the important fact that Cache Storage is distinct from the HTTP cache. [27]
HTTP Archive, "Correlating Lighthouse scores to page-level CrUX data"	Original empirical analysis/data query	Concrete large-scale comparison showing that lab Lighthouse outcomes and field CrUX outcomes are associated but far from interchangeable; retain its historical metric-version caveats. [49]
Steiner & Gao, "What slows you down? Your network or your device?"	Original research paper	Strong motivation for modeling compute/device capability separately from connectivity; reports large device-generation effects on observed mobile performance. [18]
Rajiullah et al., "Web Experience in Mobile Networks: Lessons from Two Million Page Visits"	Peer-reviewed original measurement study	Large operational-mobile-network evidence base for treating real network conditions as heterogeneous and more complex than generic cellular labels. [23]
Sengupta et al., "Through the Lens of Google CrUX: Dissecting Web Browsing Experience Across Devices and Countries"	Peer-reviewed/ original CrUX analysis	Direct quantitative evidence for device- and geography-stratified field distributions; useful when designing target strata. [12]
Goel et al., "Domain-Sharding for Faster HTTP/2 in Lossy Cellular Networks"	Original research paper	Useful methodological example of taking characteristics of cellular conditions seriously and evaluating controlled behavior under lossy conditions rather than relying on bandwidth labels alone. [24]
Demir et al., "Reproducibility and Replicability of Web Measurement Studies"	Peer-reviewed web-measurement methodology	Important methodological source for documenting browser/crawler configuration and recognizing experimental-setup sensitivity in web measurements. [38]
Linux `netem` documentation	Primary network-emulation implementation documentation	Defines packet-level delay, loss, jitter, reordering, duplication, and rate mechanisms; useful implementation reference for a packet-shaped agent mode. [50]
RIPE Atlas documentation and measurement corpus	Primary distributed-network measurement infrastructure/data	Supports selecting and reasoning about distributed probes by network path, IP, ASN, and country rather than geography alone. [8]

Priority source	Type	Why it matters to the profiler
Heričko, Šumak & Brdnik, "Towards Representative Web Performance Measurements with Google Lighthouse"	Original research paper	Directly addresses Lighthouse measurement variability and the risks of treating individual synthetic runs as representative; particularly relevant to run-count and reporting policy. [51]

Taken together, the evidence supports a profiler architecture based on **explicit conditional claims rather than simulated-user claims**. The strongest future agent strategy would make the distinction between *observed field distribution*, *selected synthetic profile*, *execution mechanism*, and *measured production outcome* visible in both the data model and user-facing terminology. RUM can provide empirical distributions and weights; reference devices and network measurements can calibrate execution; production targets supply genuine deployed-system behavior; and controlled repetition supplies causal leverage. None of those steps converts the resulting scripted visits into RUM, but their combination can make synthetic performance experiments substantially more representative, interpretable, and experimentally defensible. [52]

---

[1] [2] [6] [7] [44] [52] Why lab and field data can be different (and what to do about it) | Articles | web.dev
https://web.dev/articles/lab-and-field-data-differences

[3] HTTP: Primer on Web Performance - High Performance Browser Networking (O'Reilly)
https://hpbn.co/primer-on-web-performance/

[4] [13] [22] [25] [33] Dimensions | Chrome UX Report | Chrome for Developers
https://developer.chrome.com/docs/crux/methodology/dimensions

[5] [29] [30] [32] [41] [45] CrUX API | Chrome UX Report | Chrome for Developers
https://developer.chrome.com/docs/crux/api

[8] Khipu Visualisation | RIPE Atlas Documentation
https://atlas.ripe.net/docs/tools-and-code/khipu/

[9] [17] [31] Performance features reference | Chrome DevTools | Chrome for Developers
https://developer.chrome.com/docs/devtools/performance/reference

[10] How To Think About Speed Tools | Articles | web.dev
https://web.dev/articles/speed-tools

[11] [49] Correlating Lighthouse scores to page-level CrUX data - Analysis - HTTP Archive
https://discuss.httparchive.org/t/correlating-lighthouse-scores-to-page-level-crux-data/2164

[12] Dissecting Web Browsing Experience Across Devices and ...
https://arxiv.org/html/2308.06409v2?utm_source=chatgpt.com

[14] [16] [35] [40] [43] [47] More accurate DevTools performance debugging using real-world data | Blog | Chrome for Developers
https://developer.chrome.com/blog/devtools-grounded-real-world

[15] Chrome DevTools Protocol - Emulation domain
https://chromedevtools.github.io/devtools-protocol/tot/Emulation/

18 What slows you down? Your network or your device?
https://arxiv.org/abs/1603.02293?utm_source=chatgpt.com

19 48 Chrome DevTools Protocol - Network domain
https://chromedevtools.github.io/devtools-protocol/tot/Network/

20 50 tc-netem(8) - Linux manual page
https://man7.org/linux/man-pages/man8/tc-netem.8.html

21 36 lighthouse/docs/throttling.md at main · GoogleChrome/lighthouse · GitHub
https://github.com/GoogleChrome/lighthouse/blob/main/docs/throttling.md

23 Web Experience in Mobile Networks: Lessons from Two ...
https://dl.acm.org/doi/10.1145/3308558.3313606?utm_source=chatgpt.com

24 [1707.05836] Domain-Sharding for Faster HTTP/2 in Lossy Cellular Networks
https://arxiv.org/abs/1707.05836

26 RFC 9111 - HTTP Caching
https://httpwg.org/specs/rfc9111.html

27 Service Workers Nightly
https://www.w3.org/TR/service-workers/

28 34 39 42 Navigation types now available in CrUX | Blog | Chrome for Developers
https://developer.chrome.com/blog/crux-navigation-types

37 lighthouse/docs/lantern.md at main · GoogleChrome/lighthouse · GitHub
https://github.com/GoogleChrome/lighthouse/blob/main/docs/lantern.md

38 Reproducible, Accurate, and Archivable Web Measurements
https://arxiv.org/html/2501.15911v4?utm_source=chatgpt.com

46 CrUX on BigQuery | Chrome UX Report | Chrome for Developers
https://developer.chrome.com/docs/crux/bigquery

51 Towards Representative Web Performance Measurements ...
https://www.researchgate.net/publication/354558629_Towards_Representative_Web_Performance_Measurements_with_Google_Lighthouse?utm_source=chatgpt.com